TP 2

Timer et librairie haut-niveau

L'objectif de ce second TP est de se familiariser avec l'utilisation de librairie dite haut-niveau, c'est-à-dire qui essaie de s'abstraire partiellement du matériel pour gagner en expressivité. Pour cela, nous allons utiliser une carte Nucleo-F103RB et la librairie HAL (*Hardware Abstraction Layer*).
Documentation carte (UM1724, 91p) : `https://www.st.com/en/evaluation-tools/nucleo-f103rb.html`
Documentation librairie (UM1850, 1208p) : `https://www.st.com/en/embedded-software/stm32cubef1.html`
Documentation microcontrôleur (RM0008, 1136p) : `https://www.st.com/en/microcontrollers-microprocessors/stm32f103rb.html`

2.1 Préparation individuelle

Le but de la préparation est de montrer votre méthodologie de recherche des informations afin de pouvoir l'optimiser par la suite. Ainsi, pour chaque item, vous développerez quels mots clés sont importants pour votre recherche et quelles méthodes vous avez utilisé (table des matières, ctrl+f, moteur de recherche, IA..., etc.) :

- À partir de la documentation de la carte, identifier la série, le produit et la taille de la mémoire Flash. De même, identifier la référence exacte du microcontrôleur.
  Note : informations disponibles dans les 10 premières pages
- À partir de la documentation du microcontrôleur, combien de timers sont disponibles sur le microcontrôleur ? Quelle information précédemment extraite vous permet d'extraire cette donnée ? Indiquez clairement le chapitre et numéro de page d'où est extrait chaque information.
  Note : limitez vous au titre des chapitres et à la première page de chaque chapitre
- Quelle est la taille du prescaler et du registre de comptage du timer 2 ? Combien y a t-il de registres de contrôle pour le timer 2 ?
- À partir de la documentation de la librairie HAL, à quoi correspondent chacun des champs de la structure `TIM_Base_InitTypeDef` ? *Note : les chapitres sont par ordre alphabétique. Utilisez la table des matières pour naviguer rapidement dans le document*
- À quels champs de la structure précédente correspondent les bits `CKD`, `ARPE`, `CMS` et `DIR` du registre `TIM2_CR1` ? *Note : un champs de la structure peut recouvrir plusieurs groupes de bits du registre matériel et réciproquement*
- Le microprocesseur fonctionne par défaut à 64 MHz. Calculer les paramètres `ClockDivision`, `Prescaler` et `Period` pour obtenir une fréquence de comptage de 10 kHz et une fréquence d'interruption de 1 Hz.
  Note : plusieurs valeurs possibles et le produit des valeurs est constant
- La carte dispose d'une LED contrôlable par l'utilisateur. Identifier le pin sur lequel elle est branchée.
- À partir de la documentation de la librairie HAL et du chapitre GPIO, à quoi correspondent chacun des champs de la structure `GPIO_InitTypeDef`. En déduire les paramètres pour configurer le pin de la LED en sortie Push-Pull.

2.2 Prise en main de STMCubeIDE

L'IDE de ST est basé sur Eclipse. À chaque fois qu'il vous demande pour paramétrer une connexion, dites non (il a du mal avec le consentement et insistera...).

2.2.1 Création d'un projet

L'IDE se lance depuis un terminal avec la commande `nohup /opt/st/STMCubeIDE/stm32cubeide &`. Vous pouvez fermer le terminal.

Pour créer un projet :

- File > New > STM32 Project
- Aller dans l'onglet Board Selector et chercher la carte F103RB. Sélectionner la et faire Next
- Donner un nom à votre projet et faire Next
- Cliquer sur le lien `Firmware Update`, puis faire Browse et mettre le chemin `/opt/st/fw`. Faire Apply & Close

— Changer la version du firmware en 1.8.0 et faire Finish
— Fermer l'onglet `.ioc` et ouvrir le fichier Core > Src > main.c

2.2.2 Raccourcis claviers

Vous pouvez utiliser l'autocomplétion en faisant Ctrl + Space. Lorsque vous avez choisi une complétion avec Enter, vous pouvez naviguer dans les champs préremplis avec ⇆. De même, vous pouvez aller à la définition d'un symbole en faisant Ctrl+click.

2.3 Timer HAL

Nous allons désormais implémenter le timer 2 avec une période de 1 seconde :
— Configurer le pin de la LED en sortie Push-Pull :
1. Configurer la structure GPIO_InitTypeDef conformément à la préparation.
2. Appeler la fonction `HAL_GPIO_Init`
3. Vérifier en faisant clignoter la LED avec une attente logicielle. Utilisez la fonction `HAL_Delay`.

— Dans le fichier Core > Inc > stm32f1xx_hal_conf.h, décommenter la ligne
`/*#define HAL_TIM_MODULE_ENABLED   */`
— Suivre ce qui est indiqué dans la section "How to use this driver" :
1. Implémenter la fonction `HAL_TIM_Base_MspInit` pour initialiser les ressources (autres que le timer lui-même) dont dépend le timer. Ici, il n'y a rien à faire car nous n'utilisons aucune ressource externe.
2. Autoriser l'horloge du périphérique (et non du timer lui-même) pour la configuration en appelant la macro `__HAL_RCC_TIMx_CLK_ENABLE();`
3. Configurer l'horloge. Ici, il n'y a rien à faire, car par défaut, c'est l'horloge du système qui est utilisée.
4. Configurer le timer en utilisant la structure précédemment étudiée (`TIM_Base_InitTypeDef`) et en appelant la fonction `HAL_TIM_Base_Init`.
5. Lancer le timer en appelant l'une des fonctions permettant une interruption.

— La librairie HAL utilise des *callbacks* (rappels) pour gérer les interruptions. L'interruption peut être trouvée dans la table des interruptions définie dans le fichier Core > Startup > startup_stm32f103rbtx.s. Implémenter une surcharge dans le fichier Core > Src > stm32f1xx_it.c et appeler la fonction de *callback* `HAL_TIM_IRQHandler`.
— La fonction `HAL_TIM_IRQHandler` distribue les interruptions en fonctions des paramétrages précédents et appelle ainsi, dans notre cas, la fonction `HAL_TIM_PeriodElapsedCallback`. Implémenter cette fonction et faites clignoter la LED.
— Enfin, il faut autoriser l'interruption dans le contrôleur d'interruption (NVIC) en utilisant les fonctions `HAL_NVIC_SetPriority` et `HAL_NVIC_EnableIRQ` dont le premier (mais pas nécessairement unique) paramètre est le numéro d'interruption. Ici, il vaut `TIM2_IRQn`.
— Vérifier le fonctionnement.

2.4 Timer en mode Input Capture

Nous allons faire l'implémentation de l'exercice 1 du TD3 pour mesurer la fréquence et le rapport cyclique d'un signal carré périodique. Pour cela, nous allons utiliser le canal 1 du timer 2.
— À partir de la documentation du microcontrôleur section 9.3.7, et de la documentation de la carte, identifier les possibles pins utilisables sur la carte. *Note : pin de la carte ET pin du microcontrôleur*
— Configurer le pin identifié en mode fonction alternative et en entrée. Régler le paramètre de vitesse sur rapide.
— Configurer la structure `TIM_Base_InitTypeDef` pour avoir la plus haute fréquence d'incrément possible, sans débordement du timer sur une période de 2 ms.
— Configurer la structure `TIM_IC_InitTypeDef` pour détecter uniquement les fronts montants. À quoi sert le paramètre `ICFilter`?
— Appeler les fonctions `HAL_TIM_IC_Init` et `HAL_TIM_IC_ConfigChannel` pour configurer le timer en mode Input Capture.

— Implémenter la fonction `HAL_TIM_IC_CaptureCallback` qui est déclenchée à chaque capture et calculer la fréquence du signal.
— Autoriser l'interruption en utilisant les fonctions `HAL_NVIC_SetPriority` et `HAL_NVIC_EnableIRQ` dont le premier (mais pas nécessairement unique) paramètre est le numéro d'interruption (`TIM2_IRQn`).
— Lancer le timer avec la fonction `HAL_TIM_IC_Start_IT` et vérifier le comportement. Pour afficher la fréquence, vous pouvez l'envoyer sur l'UART et la lire sur le PC. Utilisez le code suivant :

```
char test[64];
sprintf(test, "mon message avec une donnee %d\r\n", madonnee);
HAL_UART_Transmit(&huart2, test, strlen(test), 100);
```

Dans un terminal sur le PC, lancez la commande : `screen /dev/ttyACM0 115200`
— Gérer les débordements avec l'interruption `HAL_TIM_PeriodElapsedCallback`. Pour activer cette interruption, appeler la macro `__HAL_TIM_ENABLE_IT` avec en paramètre votre *handle* et l'interruption de mise à jour (*update*).

2.5 Pour aller plus loin

Vous pouvez ajouter le calcul du rapport cyclique à votre programme. Pour cela, il faut détecter les fronts montants et descendants. Adaptez votre code en conséquence.

Le fonctionnement en mode Input Capture utilise beaucoup d'interruptions si on a un signal haute fréquence. Il est possible de s'en passer en utilisant le DMA pour sauvegarder automatiquement les temps des fronts et de calculer périodiquement la fréquence.

Un peu de lecture pour constater que la librairie HAL est limitante :

`https://community.st.com/t5/stm32-mcus-products/trouble-catching-overflow-interrupt-period-elapsed-on-tim3-while/td-p/74402`

On peut également constater que TIM5 n'est pas disponible dans HAL mais existe dans la RM0008...